

Multipliers

MDE - Advanced Engineering Topics, ITESO

Gomez Enriquez, Gerardo

gerardo.gomez@iteso.mx

Guadalajara, Mexico

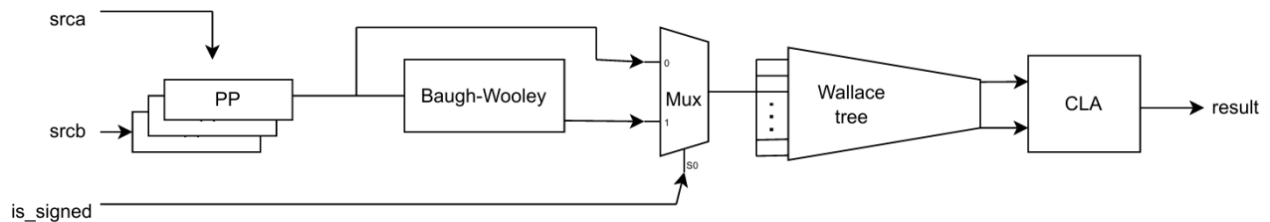
Architecture and considerations

Array-Wallace multiplier

This architecture consists of an array multiplier with the modified matrix of partial products (PP) to handle two's complement natively using Baugh-Wooley; and a Wallace tree of Carry Save Adders (CSA) and a Carry Look-ahead Adder (CLA) acting as the CPA to collapse the result to add PPs.

Considerations: Using Baugh-Wooley adds an extra partial product for the constant '1' to complete the partial products correction as per the MSBs.

Width (A×B)	Partial Products	Wallace Tree Operands (PP + corr)	Wallace CSA Levels	CLA LCU Levels
8×8	8	9	4	2
32×32	32	33	8	3
64×64	64	65	10	4
45×72	72	73	10	4

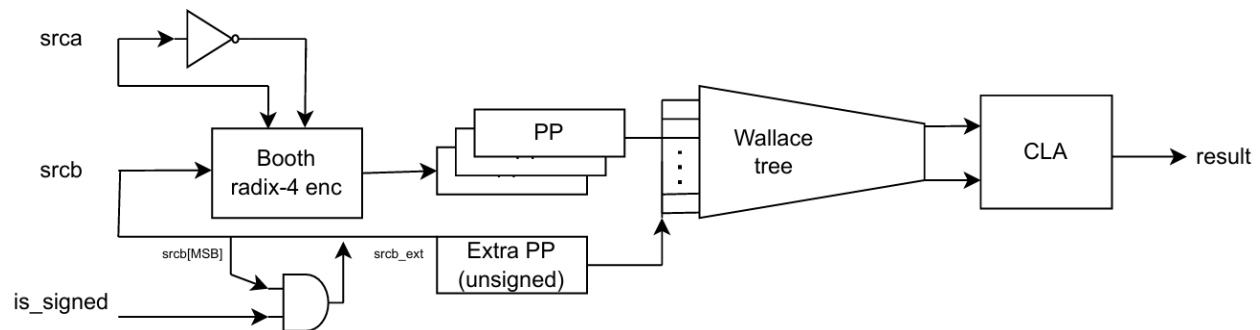


Booth radix-4 Wallace multiplier

The other multiplier also adds the partial products using a Wallace tree and a CLA to collapse the result but generates the partial products using Booth radix-4 encoding. The main difference between the multiplier architectures is how partial products are generated (Booth radix-4 vs traditional PP array) and how signed/unsigned multiplications are handled (Natively in Booth but with sign extension vs using Baugh-Wooley).

Considerations: Booth encoding natively supports signed operands. To support unsigned operands, source B width is extended 1-bit which is a constant 0 in the extra MSB. This increases the number of PPs by one. Also, another Wallace tree operand is needed to correct the two's complement of the negative PPs (use one's complement in the negative PPs and add constant '1's in the LSB positions of the negative PPs).

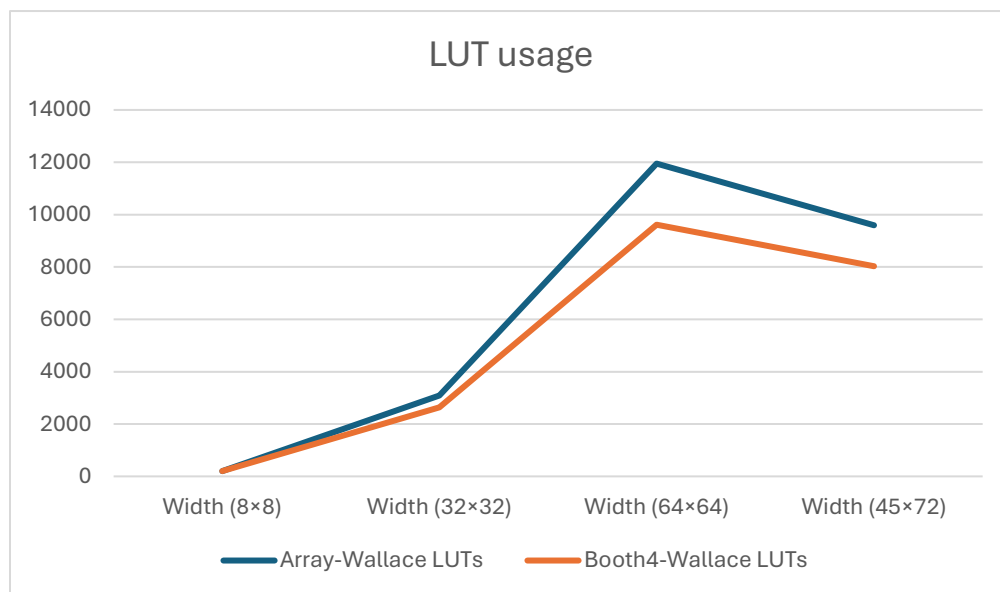
Width (A×B)	Partial Products (extra PP for unsigned)	Wallace Tree Operands (PP + comp2_corr)	Wallace CSA Levels	CLA LCU Levels
8×8	5	6	3	2
32×32	17	18	6	3
64×64	33	34	8	4
45×72	37	38	8	4



Area / resource usage analysis

LUT usage

Width (A×B)	Array-Wallace LUTs	Booth4-Wallace LUTs	Booth4 reduction vs Array
8×8	201	202	+0.5% (negligible)
32×32	3,093	2,638	-14.7%
64×64	11,950	9,615	-19.5%
45×72	9,592	8,034	-16.2%

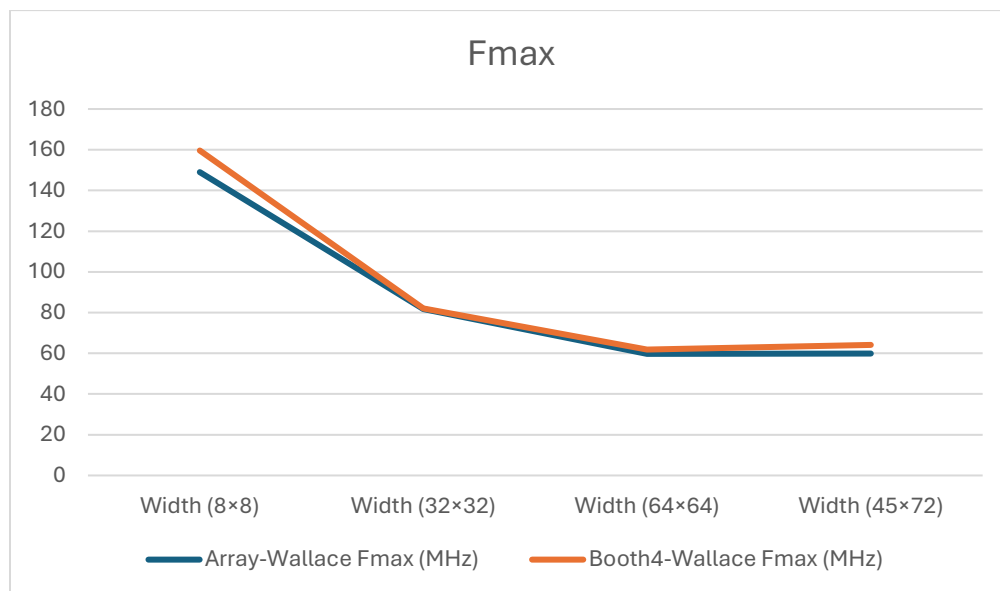


- At 8×8 the two architectures are essentially identical in area — Booth radix-4 halves the number of partial products but the encoding logic adds a comparable overhead at this narrow width.
- From 32×32 onward, Booth radix-4 consistently uses fewer LUTs because reducing the partial product count from N to $N/2$ pays off more than the Booth encoder overhead.
- Area reduction in Booth radix-4 is limited to ~15-20% due to: The partial products that are wider to support unsigned multiplications; the sign extension overhead (sign-extension reduction trick was not implemented but the array version uses Baugh-Wooley to handle signed numbers); and the implementation technology (in FPGA we don't see finer grained differences — using LUTs vs dedicated / optimized standard cells such as in ASIC).

Timing / latency analysis

Fmax

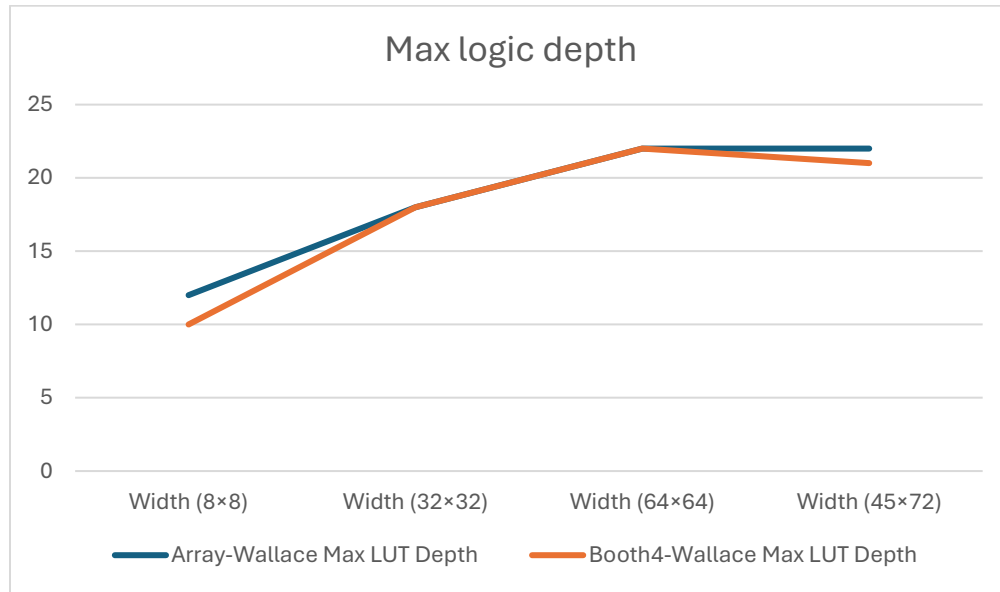
Width (A×B)	Array-Wallace Fmax (MHz)	Booth4-Wallace Fmax (MHz)	Δ Fmax (MHz)
8×8	148.99	159.62	10.63
32×32	81.7	81.97	0.27
64×64	59.73	61.84	2.11
45×72	59.88	64.08	4.2



- Booth4-Wallace is marginally faster across all widths. Both architectures converge to a similar Fmax in the 60–82 MHz range for wider operands because the critical path is dominated by the Wallace tree CSA levels and the final CLA adder, which are identical between the two implementations.
- The Booth radix-4 encoder halves the number of partial products (fewer CSA levels), but Wallace tree does not scale linearly with the number of partial products, so it gets a slight improvement in logic depth.

Max logic depth

Width (A×B)	Array-Wallace Max LUT Depth	Booth4-Wallace Max LUT Depth	Δ Depth
8×8	12	10	-2
32×32	18	18	0
64×64	22	22	0
45×72	22	21	-1



- Booth4-Wallace has equal or shallower logic depth in all cases, consistent with its higher Fmax. The improvement is only visible at 8×8 (−2 LUTs) and 45×72 (−1 LUT); at 32×32 and 64×64 the depths are identical, which explains why the Fmax difference there is nearly negligible.
- The dominant factor at wider widths is the final CLA adder depth, which is shared between both architectures and sets a floor that Booth's reduced partial-product count can no longer improve.

Critical paths

Array-Wallace Multiplier – Critical paths

Width	From Node	To Node	Depth	Comments
8×8	srca_q[0]	result_q[15]	12	<i>To</i> node is the MSB of the result. <i>From</i> node is the lowest srca index
8×8	srca_q[5]	result_q[15]	12	Same destination — all 8×8 paths converge at result MSB
8×8	srcb_q[1]	result_q[15]	12	Both srca and srcb lower indices contribute symmetrically
32×32	srcb_q[4]	result_q[55]	18	Destination is upper-middle bit (55/63); low srcb index travels furthest through Wallace tree
32×32	srca_q[20]	result_q[55]	18	Same destination; mid-range srca index also critical
32×32	—	—	—	Only 2 unique paths at the max depth in the 100-path report
64×64	srcb_q[5]	result_q[107]	22	Low srcb index dominates in the <i>From</i> node. Highest bits of the result in the <i>To</i> node.
64×64	srcb_q[5]	result_q[125]	22	Same source, different higher upper-range destinations
64×64	srca_q[43]	result_q[125]	22	Upper-mid srca index also reaches high-index result bits
45×72	srca_q[7]	result_q[91]	22	All paths target result_q[91]; srca drives critical path due to asymmetric width
45×72	srca_q[25]	result_q[91]	22	Multiple srca indices funnel to same result bit
45×72	srca_q[26]	result_q[91]	22	Adjacent srca indices both critical

Booth4-Wallace Multiplier – Critical paths

Width	From Node	To Node	Depth	Comments
8×8	srcb_q[3]	result_q[11]	10	Only srcb center bit is critical — Booth triplet encoder dominates at small widths
8×8	srcb_q[3]	result_q[13]	10	Same source, multiple upper-bit result destinations — encoder output fans into several PPs
8×8	srcb_q[3]	result_q[14]	10	srca not critical here; Booth encoding is the sole bottleneck at 8×8
32×32	srcb_q[6]	result_q[55]	18	srcb[6] is a triplet center (even index); encoder decode still on critical path
32×32	srcb_q[14]	result_q[55]	18	srcb[14] another triplet center; multiple encoder stages contribute to depth
32×32	srcb_q[6]	result_q[57]	18	Same source, adjacent destination
64×64	srca_q[54]	result_q[127]	22	Only 1 unique critical path with highest logic depth found; upper srca index feeds result MSB
64×64	—	—	—	Only 1 unique path at this depth in the 100-path report
64×64	—	—	—	—
45×72	srca_q[1]	result_q[93]	21	Low srca index critical; wide srcb (72-bit) generates many PPs, deepening Wallace tree
45×72	srca_q[1]	result_q[95]	21	Same source, adjacent mid-bit result destinations
45×72	srca_q[1]	result_q[111]	21	srca[1] reaches both mid and upper result bits

Other timing parameters

Both architectures share similar parameter values:

- **Latency:** 1 cycle (single cycle)
- **Throughput:** 1 operation per cycle
- **Slack:** Significantly negative for both architectures. Frequency was set to 1 GHz for analysis purposes and getting the maximum possible frequency, which was 159.62 MHz for the Booth radix-4 with Wallace tree when widths are 8.

Conclusion

Implementing Booth radix-4 encoding made a moderately difference in terms of area. The 15–20% LUT reduction at wider widths is real and useful but is limited mainly due to FPGA LUT abstraction and the sign-extension overhead (sign-extension reduction trick was not implemented — extended sign in higher columns waste too much extra FAs in the Wallace tree).

From timing perspective adding Booth radix-4 encoding on top of a Wallace tree + CLA did not meaningfully improve Fmax. The encoding reduces the number of Wallace tree levels by one or two at most.

Width	Recommended Architecture	Rationale
8×8	Array-Wallace	Booth4 offers no area or meaningful timing benefit; encoder overhead eliminates any savings; simpler design
32×32	Booth4-Wallace	~15% area saving at negligible timing cost; justified for area-sensitive designs
64×64	Booth4-Wallace	~20% area saving with no timing penalty; clear winner when area matters
45×72	Booth4-Wallace	~16% area saving with a modest 7% Fmax improvement; best trade-off across both metrics